

Written Homework 3:

Deep Learning

Jeffrey Ochavillo

Abstract & Introduction:

In this study, we explore the application of neural networks for the identification of bird species in the Seattle area based on their sounds. Our approach utilizes processed data comprising spectrograms extracted from 10 mp3 sound clips of varying durations for each of the 12 common bird species found in the region. By taking advantage of the power of neural networks, we aim to develop a custom model capable of accurately predicting the species of a bird solely based on its audio recordings.

To accomplish this, we employ a deep learning framework that incorporates convolutional neural networks (CNNs) for spectrogram analysis and classification. The CNN architecture allows the model to learn hierarchical representations from the spectrogram data, capturing both local and global acoustic features that differentiate bird species. This enables the network to effectively discriminate between various birds by their calls or sound.

Prior to training the neural network, the spectrogram data undergoes preprocessing steps to enhance the quality and relevance of the input features. These steps involve normalization, noise reduction, and scaling to ensure consistent and optimal performance of the model. Additionally, we employ techniques such as data augmentation to expand the training set, promoting generalization and reducing overfitting.

The training process involves optimizing the model's parameters using a suitable loss function, such as categorical cross-entropy, and applying backpropagation to update the network weights. We adopt a split of the dataset into training, validation, and testing sets to evaluate the model's performance and prevent overfitting. We employ strategies like early stopping and regularization to prevent model convergence issues and enhance generalization.

Through rigorous experimentation and hyperparameter tuning, we seek to achieve a high accuracy rate in species classification. We evaluate our model using metrics such as precision, recall, and F1 score to assess its overall effectiveness. Additionally, we compare our results with existing bird identification methods to establish the efficacy of our neural network-based approach. The results may surely vary based on the model of the structure, the amount of units we decide to measure, and the data we have. In the case for the binary classification, the Blue Jay and Stellar Jay classification performed great, getting accuracies from 85% to 95% on average with low computing time. Compared to classifying on all 12 species, our accuracies dropped to around 60% to 70% but that's to be expected from our model and how vast the data is.

The successful implementation of a custom neural network for bird species identification through sound analysis has promising implications for monitoring and conservation efforts in the Seattle area. By automating the identification process, this technology could facilitate the collection of valuable bird population data, contribute to ecological research, and aid in environmental planning and decision-making.

Theoretical Background:

Convolutional Neural Networks (CNN)

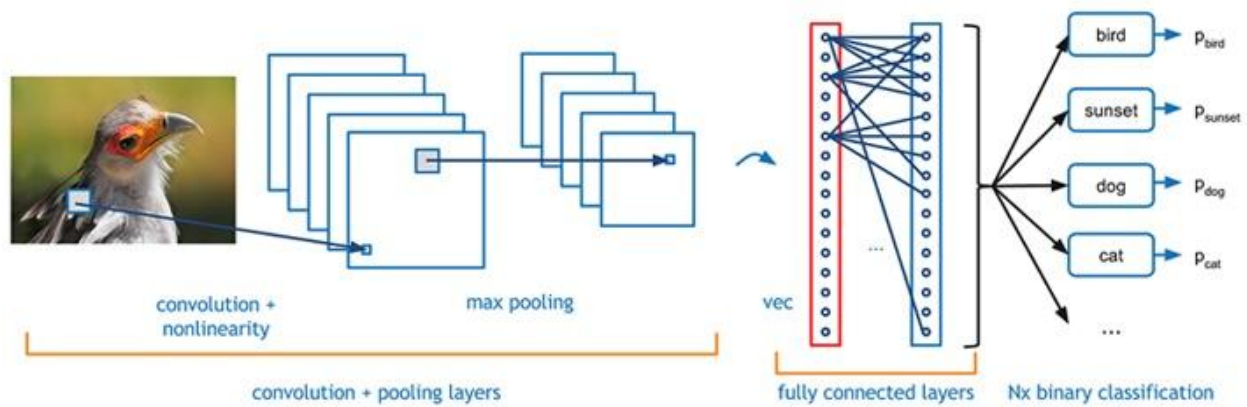
One method we will be using to classify the bird species is *Convolutional Neural Networks* (CNNs). CNNs are a class of deep learning models signed for processing grid-like data, examples are images or spectrograms in this case. The foundations lie in the utility of capturing spatial hierarchies of features and learning meaningful representations directly from raw input data.

A core characteristic or feature of CNNs are convolutional layers. The purpose of these layers is to perform local receptive field operations by convolving learned filters or kernels with input data. By doing this, the network can extract patterns and features while preserving spatial relationships. The filters are then applied throughout the input so the pattern detection is applied in different regions or parts of the input. By stacking the layer, CNNs can learn more complex and abstract representations.

After the going through convolving, pooling layers are applied, either max or average pooling to downsize the feature maps, reducing spatial dimensions while retaining the more valuable and interpretable features. By downsampling, it helps the neural network be invariant to spatial translations and reduces the computational burden by minimizing the number of parameters in subsequent layers.

A big contributor to the neural networks are the functions applied. Common activation functions applied were Rectified Linear Unit (“relu”). ReLU introduced nonlinearity by setting negative values to 0. By applying activation functions allows nonlinear variables to be inputted and increases the models’ capabilities to learn more complex patterns and representations.

Afterwards, connected layers of the CNNs are working together after convolving and pooling of the layers. What these connected layers do is they connect every node, input, or neuron to every or node, input, or neuron in the next layer. This allows the network to learn high-level abstractions and formations based on previously given or extracted features. The output layer of the CNN utilizes an activation function, example would be a softmax, to produce probability distribution over the classes for classification tasks.



When we get to training the model, backpropagation is applied. This is when gradients are computed and used to update the network's weights and biases between the predicted outputs and labels. This is used mostly on categorical cross-entropy to minimize the defined loss function. In the model we used, we applied optimization algorithms like Adam to update the parameters and find the optimal configuration.

Methodology:

The steps of processing the data were already taken care of since we had options of choosing between the raw format of the spectrograms or using data that was already preprocessed. For this experiment, the preprocessed data was ultimately chosen to avoid more tedious cleaning.

The preprocessed data came in a structure of a zip file that needed to be individually read and put into a readable data frame in the software. After loading all the processed bird

spectrogram data from the respective folders, the next step was choosing the two we wanted to experiment with for the first model. For our model, we chose to analyze the Blue Jay and Stellarjay. From a physical view, there clearly is a difference between the two birds but the sounds due differ. Having known that before going into the analysis, surely choosing birds that sound very similar would have more interesting results but these two were ultimately decided.

For a trial experiment, we implemented a CNN that consisted of two hidden layers with 64 and 32 units, each followed by a dropout later. The output layer has 2 units with softmax activation for binary classification. The first layer is a fully connected dense layer with 64 nodes with a ReLU activation function. Following that is a dropout layer that regularizes the model. The dropout sets a fraction of input units to 0 during the training phase to help with the overfitting of the data. We then added a 50% rate that let us drop 50% of the units during training. The following is another dense layer with 32 nodes and another ReLU activation function. Then another dropout layer of 50% was added and finally the output layer, with 2 units, thus represents the two classes of the binary classification. The output is a softmax, which converts the output values into probabilities for each class.

The process is the same when running on all 12 of the bird species compared to just 2 species of birds. The process took combining all the spectrograms into one single array based on where the species was located by their index. After, using the index we assigned labels for identification so we have a base to use for our training and testing data. Afterwards, we shuffled the data along with the labels.

For the architecture of the neural network, the input layer which specifies the shape of the input data had 343 frequency bins, 256 steps, and 1 channel. The input shape is then set to 343 by 256 by 1 to match all dimensions. The convolution layer contained 32 filters, each containing

a kernel size of 3x3 and the activation function used was ReLU to introduce non-linearity to the model. The max pooling layer was given a pool size of 2x2. The flattening layer was then output to be a 1 dimensional layer, and just for emphasis this is important for transition from the convolutional layer to the fully connected layers to operate properly. Following the flattening layer are the two dense layers. The first dense layer contains 64 units and uses a ReLU activation function. This layer learns higher-level features by transforming the flattening input. Then is the dropout layer that has a dropout rate of 0.5 that helps with overfitting by reducing the reliance on the features to enforce learning. Lastly the output layer has 12 units. It uses a softmax activation function, which outputs probabilities for each class.

From these model architectures, we then adjusted the models by adding more layers, increasing the amount of activation functions, input layers, convolutions, to provide us with various outputs. By making these changes, drawbacks could be that the model will not always perform as expected. Some models even with less layers or just a more simple model can produce a higher accuracy and take a lot less time. Compared to some with a very complex architecture it can vary in runtime as quick as a 1 minute to even half an hour and the accuracy is worse. This was demonstrated in my theoretical result which will be discussed next.

Computational Results:

The discussion here will go through the outputs, run times, and results that were received after running through different model structures from our CNN.

The models of Figure 1 and Figure 2, trying to classify the measure between Blue Jay and Stellar Jay birds performed pretty well. The difference between the two structures is that the first model has 2 dense layer with first layer having 64 units and the second dense layer has 32, compared to the second model containing a total of 3 dense layers, the first dense layer has 128

units, second dense layer has 64 units, and the last layer has 32 units. Changing the number of units will impact the capacity of the model and its ability to learn patterns from the inputs. But in general, increasing the units will likely increase the model's ability to learn. But the caveat is that we will be prone to overfitting which could be slightly noticed in both models. In terms of the accuracy output, the first model gave an accuracy between 88% to 94% in some runs. Compared to the second model, it gave accuracies between 78% to 88%. So interesting enough, the simplest model with the least amount of units gave a better performance in classifying which bird species was which from the inputs.

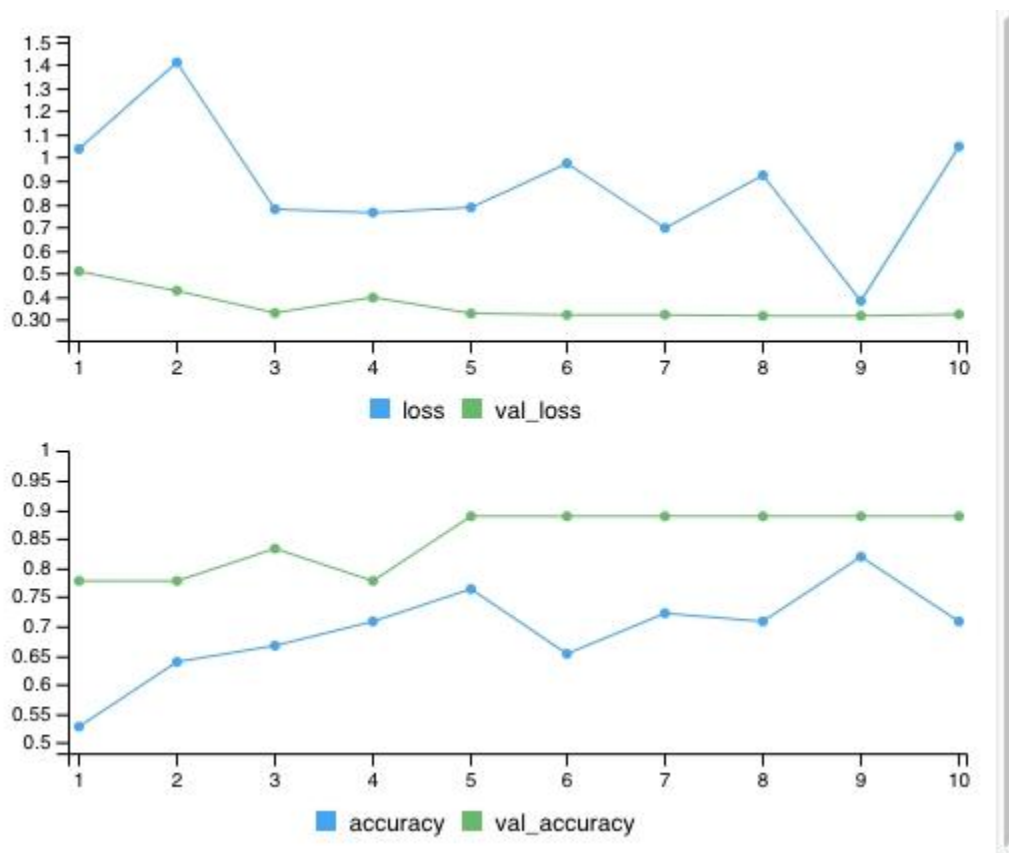


Figure 1. First Binary Classification Model Performance

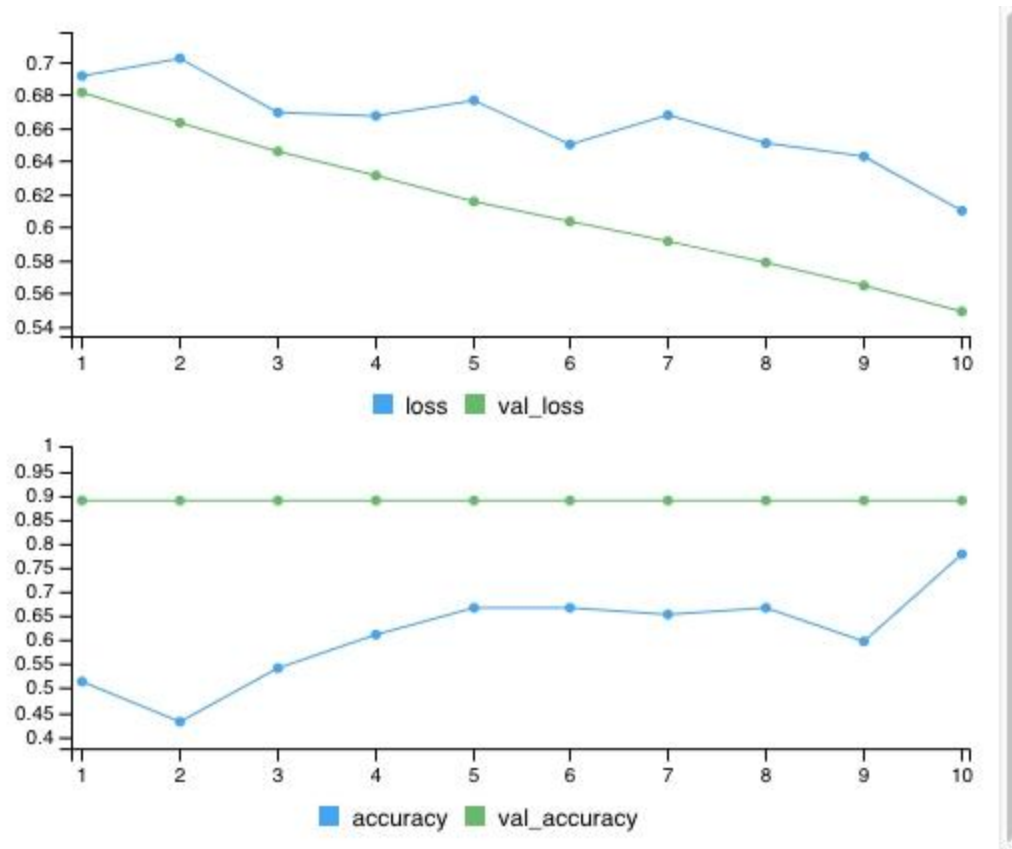


Figure 2. Second Binary Classification Model Performance

The figure below, Figure 3, is the output graph after going through the CNN model of a fairly complex architecture that we experimented with. As you can tell from the graph. The training loss is as expected by having a slope descent downwards as we go through the 20 epochs. But the testing values which are shown in green started very low and shown are a few jumps from the 6th to the 14th epoch. As for our accuracy, our test accuracy shows that is it went through the 20 epochs, there was an upward slope. However, when it came to the true values in green, the values stayed consistently low in the valuation line. This could be seen since the accuracy resulted was 0.1091954 or 10.91%.

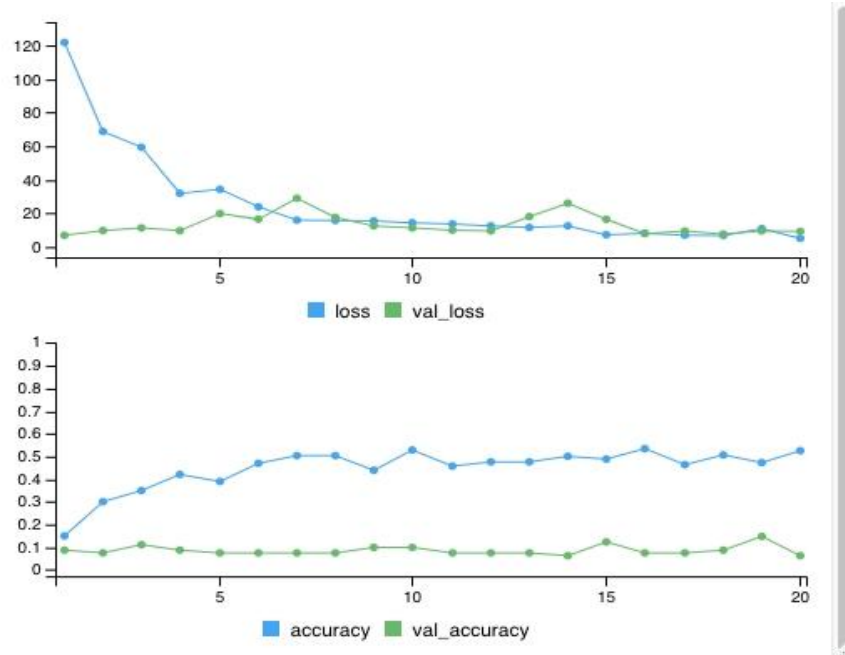


Figure 2. First Model for Multiclass CNN

The next model actually performed better despite not having a complex model structure and took less of a run time. This model can be seen below in Figure 4. It performed significantly better. You can clearly see the points from the training and testing from the loss function to be nearly the same. In addition, the training and testing values for measuring accuracy were not completely together but they followed the same trend and the values were not too far apart. Potentially there was overfitting in the measure but the accuracy for this simpler model came out to be around 65%.

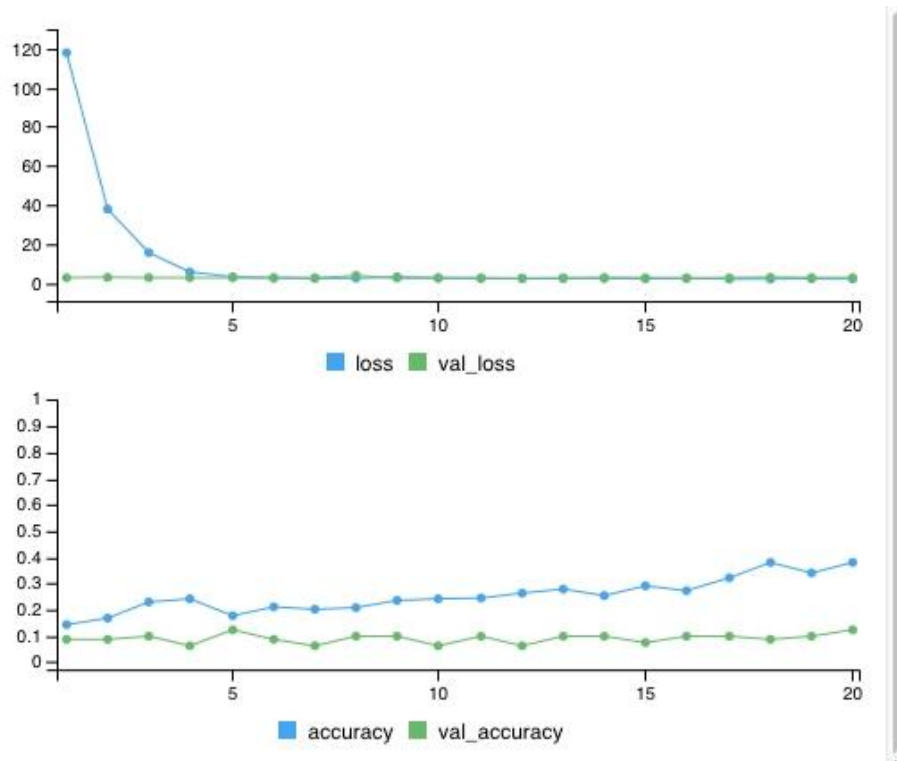


Figure 4. Second Multi Classification CNN

Discussion:

The limitations I experienced was working with an entirely different structured dataset. Working on datasets containing images and regular column structured data such as baseball averages is slightly different than working with spectrograms or audio data of different bird species. It was difficult because knowing how to reshape and structure your data from the start is key in order to get the models to read and go through the entire neural network process to calculate the right accuracies.

From listening to samples of the Blue Jay and Stellarjay birds, the Blue Jay gives a more soft and subtle sound. It can be similar to the sound of a bell possibly. However, the Stellarjay on the other hand gives a more sharp and harsher tone of sound. So in a way, it could be distinguished between the two bird species on which birds make a certain sound if heard. But if I

were to rerun the experiment, trying to find bird sounds that sound very much alike and run the CNN algorithm would be something I would look into in the future.

Another model we could have used was the Recurrent Neural Networks (RNNs). The reason for saying this is because RNNs are suitable for sequential data such as the bird spectrogram data we are currently using. RNNs can capture temporal dependencies and are useful for tasks like speech recognition. But in this case, RNNs could be used with time-series data as well.

The reason why we stuck to using CNNs is because they can identify features regardless of their location in the input, can capture local patterns and features in images, and can set parameters for shared learning. In this case with the bird spectrograms, as the position of bird vocalizations can vary, CNNs can utilize relevant features of the audio to possibly find the location in the whole recording. Since the data can vary in dimension, CNNs work well at learning and recognizing frequency patterns or structures that indicate and classify which bird that could be. Lastly, with all the extra learning filters we can add in the model, this can help reduce the number of parameters in the network and capture patterns effectively. In this case, we can identify key frequency points across the whole recording and identify patterns and add filters throughout the model to potentially better our classification of the bird.

Conclusions:

Overall, taking advantage of CNNs and other deep learning methods was quite interesting and does take up a lot of computing power. We can surely expect that when we dive into other formats of data that are more complicated, expect the run time to increase along with the model complexity. But in the case of the bird spectrograms from different bird species in Washington, after extracting all the necessary data after some data transformations, running the CNN model

worked great. The accuracies from the binary class between Blue Jay and Stellar Birds came out to be high probably due to the very distinct sounds they give. Possibly changing the bird species to look into bird species that sound similar and applying our models can truly measure the accuracy of CNN. As for the multiclass, looking into modifying our structure can surely improve our accuracy but just know that it will take a lot of processing time due to all the training it has to go through for all 12 species.

Bibliography/References:

- Vopani. (2020, September 14). Xeno-Canto Bird Recordings Extended (A-M). Kaggle. <https://www.kaggle.com/datasets/rohanrao/xeno-canto-bird-recordings-extended-a-m>
- dshahid380. (2019, February 26). Convolutional Neural Network . Medium. <https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2022). An introduction to statistical learning: With applications in R. Springer.

Appendix:

Written_Homework3

Jeffrey Ochavillo

2023-05-10

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.2      ✓ readr      2.1.4
## ✓ forcats    1.0.0      ✓ stringr    1.5.0
## ✓ ggplot2    3.4.2      ✓ tibble     3.2.1
## ✓ lubridate  1.9.2      ✓ tidyr      1.3.0
## ✓ purrr      1.0.1
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
library(keras)
library(e1071)
library(dplyr)
library(abind)
```

Load the data

```

amecro <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/amecro.dat")
amecro=species
barswa <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/barswa.dat")
barswa=species
bkcchi <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/bkcchi.dat")
bkcchi=species
blujay <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/blujay.dat")
blujay=species
daejun <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/daejun.dat")
daejun=species
houfin <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/houfin.dat")
houfin=species
mallar3 <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/mallar3.dat")
mallar3=species
norfli <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/norfli.dat")
norfli=species
rewbla <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/rewbla.dat")
rewbla=species
stejay <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/stejay.dat")
stejay = species
wesmea <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/wesmea.dat")
wesmea=species
whcspa <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/whcspa.dat")
whcspa=species

```

```
library(dplyr)
```

```

w3_data <- abind(amecro, barswa, bkcchi, blujay, daejun, houfin, mallar3, norfli, rewbl
a, stejay, wesmea, whcspa, along =1)
dim(w3_data)

```

```
## [1] 580 343 256
```

```

x <- abind(blujay, stejay, along = 1)
#x <- abind(x, along = 4)
x <- array_reshape(x, c(nrow(x),87808))
y <- c(rep('blujay',dim(blujay)[1]), rep('stejay',dim(stejay)[1]))
y_factor <- factor(y,level = c('blujay','stejay'))
y_encode <- to_categorical(as.integer(y_factor)-1, num_classes = 2)
# y_encode
# glimpse(y)
# dim(w3_data)
# glimpse(w3_data)
# dim(x)

```

```
set.seed(42)
library(keras)
library(abind)
w3_data <- abind(amecro, barswa, bkcchi, blujay, daejun, houfin, mallar3, norfli, rewbl
a, stejay, wesmea, whcspa, along =1)
dim(w3_data)
```

```
## [1] 580 343 256
```

```

# Combine the spectrograms into one matrix
x <- abind(blujay, stejay, along = 1)
y <- c(rep('blujay', dim(blujay)[1]), rep('stejay', dim(stejay)[1]))

# length(y)
# Create the target variable (labels)

x <- array_reshape(x, c(nrow(x), 87808))
y_factor <- factor(y, levels = c('blujay', 'stejay'))
#length(y_factor)
y_encode <- to_categorical(as.integer(y_factor) - 1, num_classes = 2)
#length(y_encode)

# Split the data into training and testing sets
n <- nrow(x)
train_measure <- sample(n, floor(0.8*n))

x_train <- x[train_measure,]
y_train <- y_encode[train_measure, ]
x_test <- x[-train_measure,]
y_test <- y_encode[-train_measure,]

# Define the model
# model <- keras_model_sequential()
# model %>%
#   layer_dense(units = 64, activation = "relu", input_shape = c(87808)) %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 32, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 2, activation = "softmax") # Output layer with softmax activation for binary classification #this model took less than 30 seconds to run

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.2810534
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.8888889

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.2052964
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.9444444

model <- keras_model_sequential()
model %>%
  layer_dense(units = 128, activation = "relu", input_shape = c(87808)) %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 64, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%

```

```

layer_dense(units = 2, activation = "softmax") # Output layer with softmax activation
for binary classification

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.3542859
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.7222222 also took less than 30 seconds to run

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.5489867
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.8888889

# Compile the model
model %>% compile(
  loss = "categorical_crossentropy",
  optimizer = "adam",
  metrics = c("accuracy")
)

# Train the model
model %>% fit(
  x = x_train,
  y = y_train,
  epochs = 10,
  batch_size = 32,
  validation_data = list(x_test, y_test)
)

# Evaluate the model
metrics <- model %>% evaluate(x_test, y_test)
cat("Test loss:", metrics[[1]], "\n")

```

```
## Test loss: 0.3532373
```

```
cat("Test accuracy:", metrics[[2]], "\n")
```

```
## Test accuracy: 0.8333333
```

classification for all 12 species

```

# Example variables to encode
variables <- c("var1", "var2", "var3", "var4", "var5", "var6", "var7", "var8", "var9",
"var10", "var11", "var12")

# Example labels (numeric)
labels <- c("amecro", "barswa", "bkcchi", "blujay", "daejun", "houfin", "mallar3", "norf
li", "rewbla", "stejay", "wesmea", "whcspa")

# Create a data frame with the variables
data <- data.frame(var1 = c(1, 2, 3),
                    var2 = c(1, 3, 2),
                    var3 = c(2, 1, 3),
                    var4 = c(3, 2, 1),
                    var5 = c(2, 3, 1),
                    var6 = c(3, 1, 2),
                    var7 = c(1, 2, 3),
                    var8 = c(2, 3, 1),
                    var9 = c(3, 1, 2),
                    var10 = c(1, 2, 3),
                    var11 = c(2, 3, 1),
                    var12 = c(3, 1, 2))

# One-hot encode the variables
encoded_data <- model.matrix(~ . - 1, data = data)

# Rename the encoded columns with labels
colnames(encoded_data) <- labels

# View the encoded data
print(encoded_data)

```

```

##   amecro barswa bkcchi blujay daejun houfin mallar3 norfli rewbla stejay wesmea
## 1      1      1      2      3      2      3      1      2      3      1      2
## 2      2      3      1      2      3      1      2      3      1      2      3
## 3      3      2      3      1      1      2      3      1      2      3      1
##   whcspa
## 1      3
## 2      1
## 3      2
## attr(,"assign")
## [1] 1 2 3 4 5 6 7 8 9 10 11 12

```

```
# Combine the spectrograms into a single array
spectrograms <- array(0, dim = c(594, 343, 256))
spectrograms[1:52,,] <- amecro
spectrograms[53:107,,] <- barswa
spectrograms[108:164,,] <- bkcchi
spectrograms[165:214,,] <- blujay
spectrograms[215:272,,] <- daejun
spectrograms[273:317,,] <- houfin
spectrograms[318:353,,] <- mallar3
spectrograms[354:412,,] <- norfli
spectrograms[413:453,,] <- rewbla
spectrograms[454:493,,] <- stejay
spectrograms[494:529,,] <- wesmea
spectrograms[530:580,,] <- whcspa

# Label the data
labels <- c(rep("amecro", 52), rep("barswa", 55), rep("bkcchi", 57), rep("blujay", 50),
            rep("daejun", 58), rep("houfin", 45), rep("mallar3", 36), rep("norfli", 59),
            rep("rewbla", 41), rep("stejay", 40), rep("wesmea", 36), rep("whcspa", 51))

# Shuffle the data and labels
index <- sample(1:length(labels))
spectrograms <- spectrograms[index,,]
labels <- labels[index]

# Create a training and testing set
train_size <- round(length(labels) * 0.7)
train_data <- spectrograms[1:train_size,,]
train_labels <- labels[1:train_size]
test_data <- spectrograms[(train_size+1):length(labels),,]
test_labels <- labels[(train_size+1):length(labels)]

dim(train_data)
```

```
## [1] 406 343 256
```

```
dim(test_data)
```

```
## [1] 174 343 256
```

```
# reshape the data
train_data <- array_reshape(train_data, c(train_size, 343, 256, 1))
test_data <- array_reshape(test_data, c(length(labels) - train_size, 343, 256, 1))

# one hot encode the labels
# train_labels <- one_hot(train_labels, 12)
# test_labels <- one_hot(test_labels, 12)

# Convert labels to numerical categories
label_categories <- factor(labels)
label_categories <- as.integer(label_categories) - 1

# One-hot encode the labels using to_categorical
train_labels <- to_categorical(label_categories[1:train_size], num_classes = 12)
test_labels <- to_categorical(label_categories[(train_size+1):length(labels)], num_classes = 12)

dim(train_data)
```

```
## [1] 406 343 256 1
```

```
dim(test_data)
```

```
## [1] 174 343 256 1
```

```

library(keras)

# Define the model architecture

model <- keras_model_sequential() %>%
  layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape =
c(343, 256, 1)) %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_flatten() %>%
  layer_dense(units = 64, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 12, activation = "softmax") #7 min run and Test accuracy: 0.643678
2

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape
= c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_flatten() %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") Test accuracy: 0.7241379

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu", input_shape
= c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 256, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_flatten() %>%
#   layer_dense(units = 256, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") #Test accuracy: 0.1091954 took 34 mi
nutes to run

#model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape =
c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%

```

```

# layer_flatten() %>%
# layer_dense(units = 64, activation = "relu") %>%
# layer_dropout(rate = 0.5) %>%
# layer_dense(units = 12, activation = "softmax") #7 min run and Test accuracy: 0.64367
82

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu", input_shape
# = c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_flatten() %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 64, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") #25 minute run with 0.05 accuracy

# Compile the model
model %>% compile(
  loss = "categorical_crossentropy",
  optimizer = optimizer_rmsprop(lr = 0.001),
  metrics = "accuracy"
)

# Train the model
history <- model %>% fit(
  train_data, train_labels,
  validation_split = 0.2,
  epochs = 20,
  batch_size = 32
)

# # Evaluate the model on test data
scores <- model %>% evaluate(test_data, test_labels)
cat("Test accuracy:", scores[[2]], "\n")

```

```
## Test accuracy: 0.6091954
```